

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory

Experiment 5

**Comprehensive Study of CNN Training, Regularization,
Optimization, Hyperparameter Tuning, Transfer Learning
and Cross-Validation**

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. The modified implementation preserves this experimental structure while changing selected parameters and improving reproducibility and memory handling. For transparency, the figures embedded below are the image outputs already present in the supplied executed notebooks; they are included as reference visuals until the modified notebooks are executed in a TensorFlow-enabled environment.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.



Figure 1: Sample images from the Oxford-IIIT Pet training data

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is suitable for CPU-based laboratory work.

Setting	Modified implementation
Dataset	Oxford-IIIT Pet via TensorFlow Datasets
Classes	37
Input size	$224 \times 224 \times 3$
Preprocessing	MobileNetV2 <code>preprocess_input</code>
Train/validation split	80% / 20% of the TFDS training split
Independent test set	TFDS test split; reserved for final evaluation
Random seed	123
Default batch size	24
Training epochs	3 per controlled experiment

4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;

- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:

Input ($224 \times 224 \times 3$) $\rightarrow$ Convolution / MobileNetV2 feature extractor $\rightarrow$ Global Average Pooling $\rightarrow$ Dense $\rightarrow$ Softmax (37 classes)

The modified implementation uses ImageNet-pretrained MobileNetV2 with a new classifier containing 160 ReLU units followed by a 37-class softmax layer.

5. Modifications Made to Part 1

The following changes were made to the supplied implementation:

Component	Original setting	Modified setting
Random seed	42	123
Default batch size	32	24
Classifier dense units	128	160
Main learning rate	0.001	0.0005
L2 coefficient	0.001	0.0005
Dropout experiment	0.50	0.30
Learning-rate candidates	0.001, 0.0001	0.0005, 0.0001
Batch-size candidates	16, 32, 64	16, 24, 48
Dropout candidates	0, 0.25, 0.5	0, 0.30, 0.50
Fine-tuning depth	Last 20 layers	Last 30 layers
Fine-tuning learning rate	10^{-5}	10^{-5}

The experimental categories remain the same: baseline training, initialization, regularization, Batch Normalization, optimizer comparison, learning-rate tuning, batch-size tuning, dropout tuning, transfer learning and fine-tuning.

6. Weight Initialization

Weight initialization determines the initial values of trainable parameters. A suitable initialization strategy can improve convergence and reduce problems such as unstable activation propagation.

The four initialization strategies investigated are:

- Zero Initialization;
- RandomNormal Initialization;
- Glorot/Xavier Initialization; and
- He Initialization.

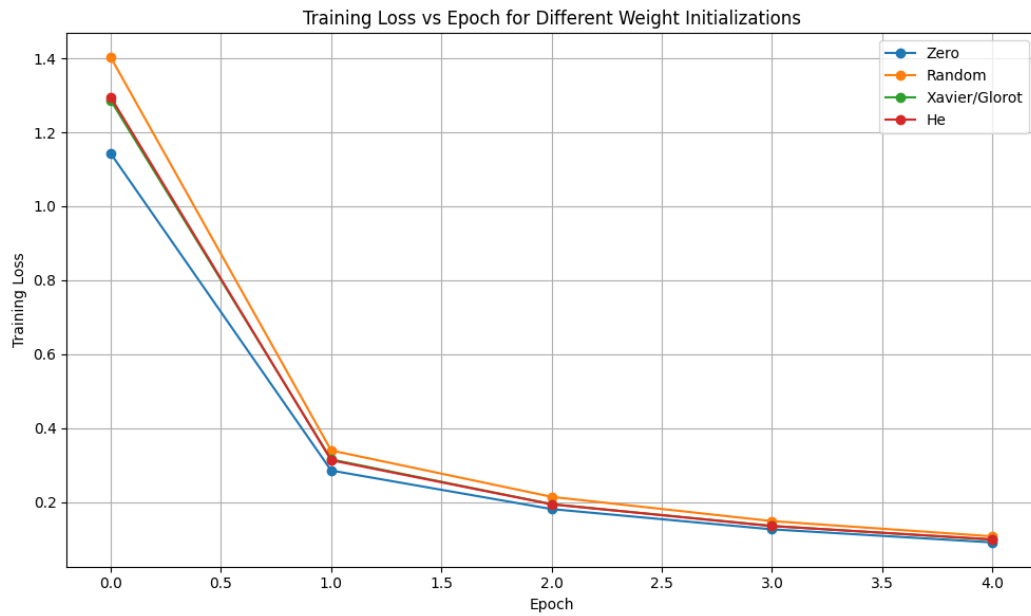


Figure 2: Training Loss for Different Weight Initialization Strategies

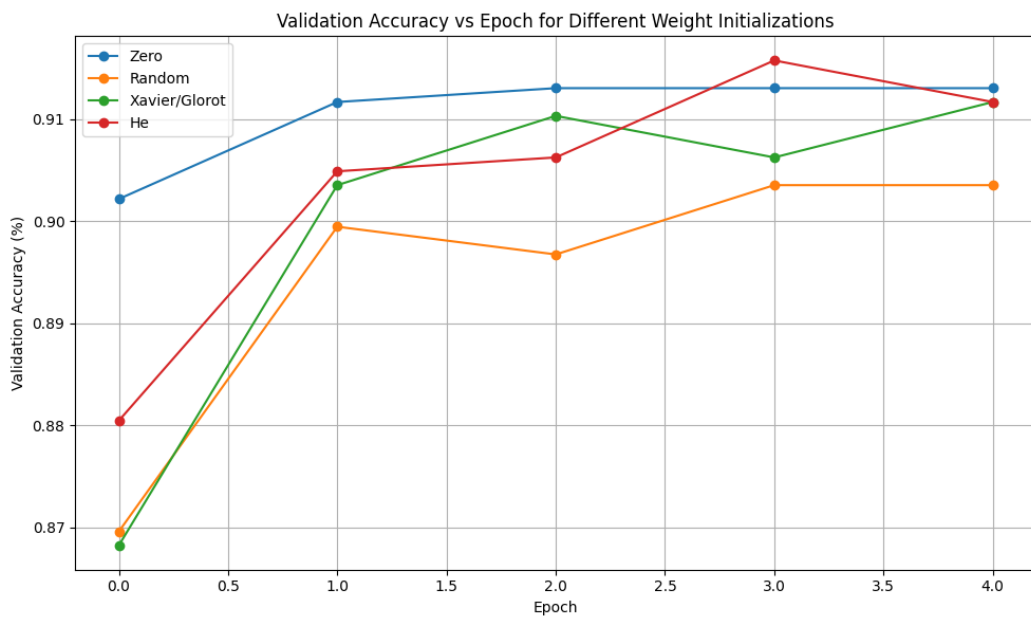


Figure 3: Validation Accuracy for Different Weight Initialization Strategies

The modified notebook computes the final training loss, best validation accuracy and training time for each initialization method. No numerical values are hard-coded; all reported values are produced by the actual run.

Inference: He initialization is particularly appropriate for ReLU-based networks because it is designed to maintain a suitable activation variance during forward propagation.

7. Regularization and Overfitting

Overfitting occurs when a model performs very well on training data but generalizes poorly to unseen data. The modified implementation compares:

- No Regularization;

- L2 Regularization with coefficient 5×10^{-4} ;
- Dropout with rate 0.30; and
- Batch Normalization.

Training and validation accuracy as well as training and validation loss are monitored.

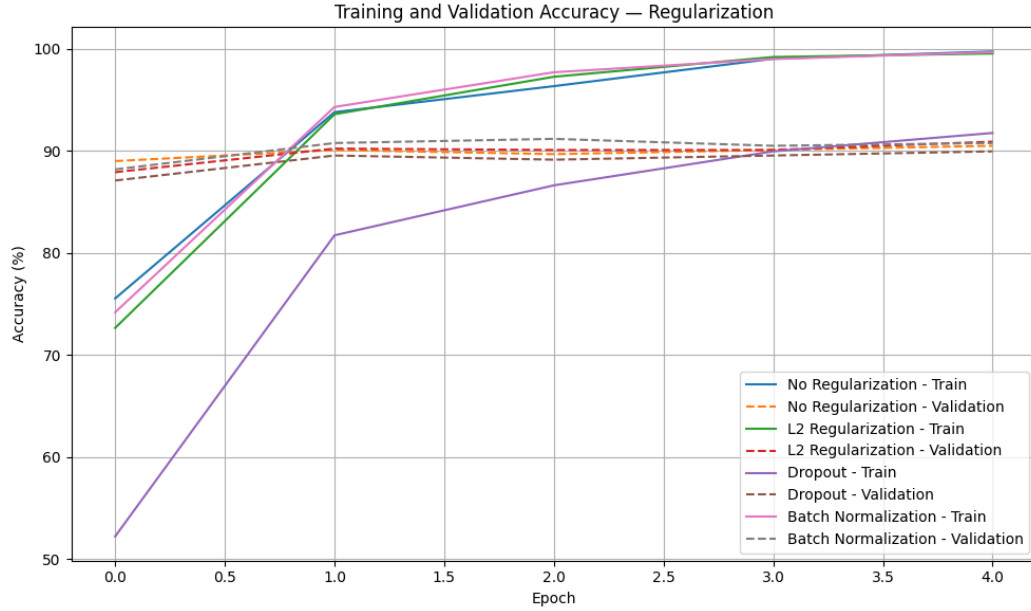


Figure 4: Training and Validation Accuracy for Different Regularization Methods

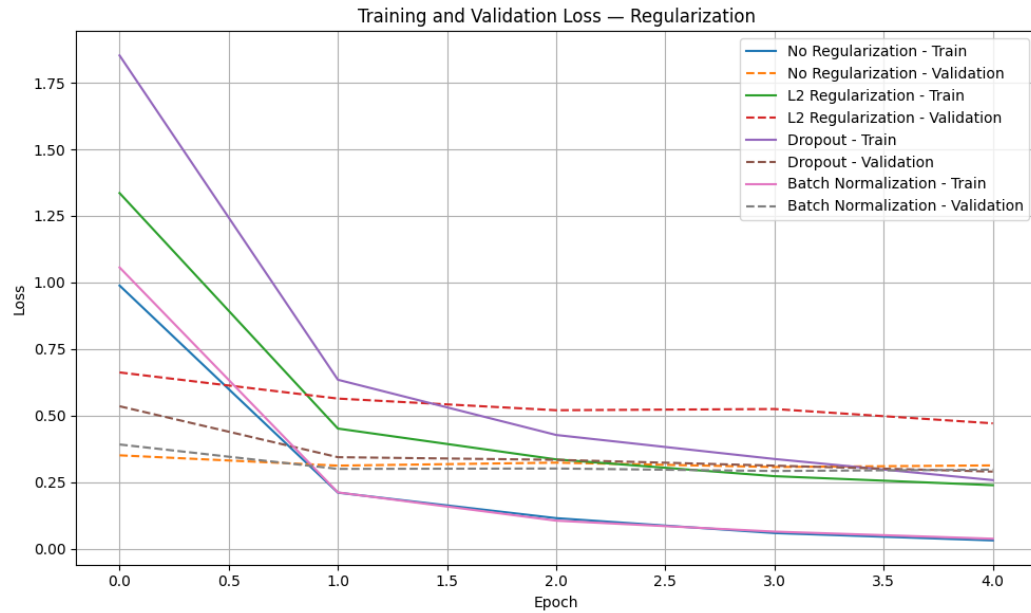


Figure 5: Training and Validation Loss for Different Regularization Methods

A model is likely to overfit when training loss continues to decrease while validation loss begins to increase. The generalization gap is examined directly from the curves rather than assumed in advance.

8. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch $x_1, x_2, \dots, x_m$:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

and the final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

For the mini-batch

$$x = [2, 4, 6, 8],$$

the mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Ignoring the very small ϵ :

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342].$$

If $\gamma = 1$ and $\beta = 0$, these become the final outputs.

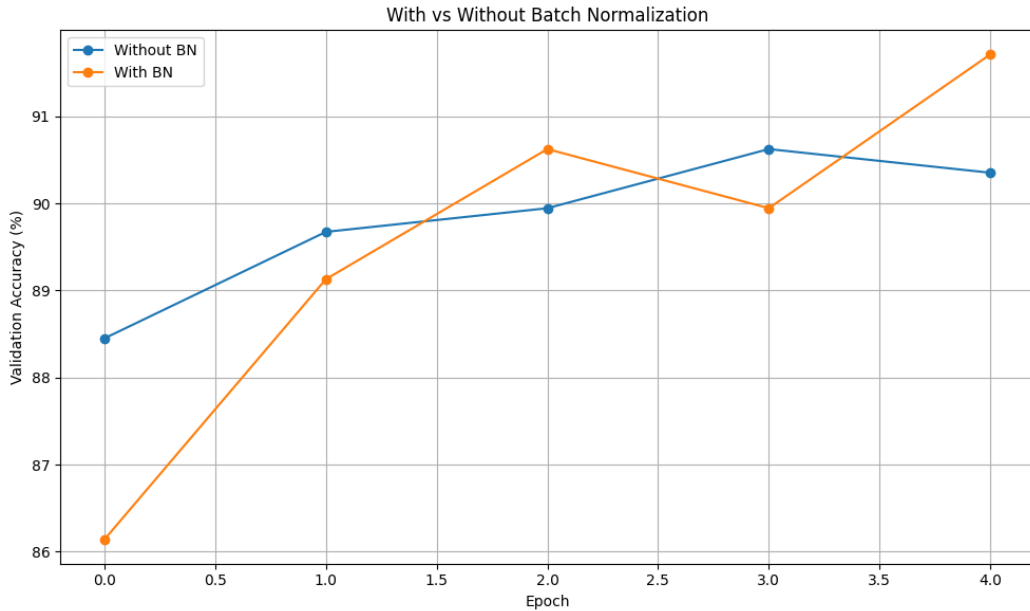


Figure 6: Validation Accuracy With and Without Batch Normalization

9. Optimization Algorithms

The following optimizers are compared under the same experimental conditions:

- Stochastic Gradient Descent (SGD);
- Momentum;
- RMSProp; and
- Adam.

The modified implementation uses a fresh optimizer instance for every model. This prevents optimizer state from being accidentally reused between independent experiments.

Optimizer	Modified setting
SGD	learning rate = 5×10^{-4}
Momentum	learning rate = 5×10^{-4} , momentum = 0.9
RMSProp	learning rate = 5×10^{-4}
Adam	learning rate = 5×10^{-4}

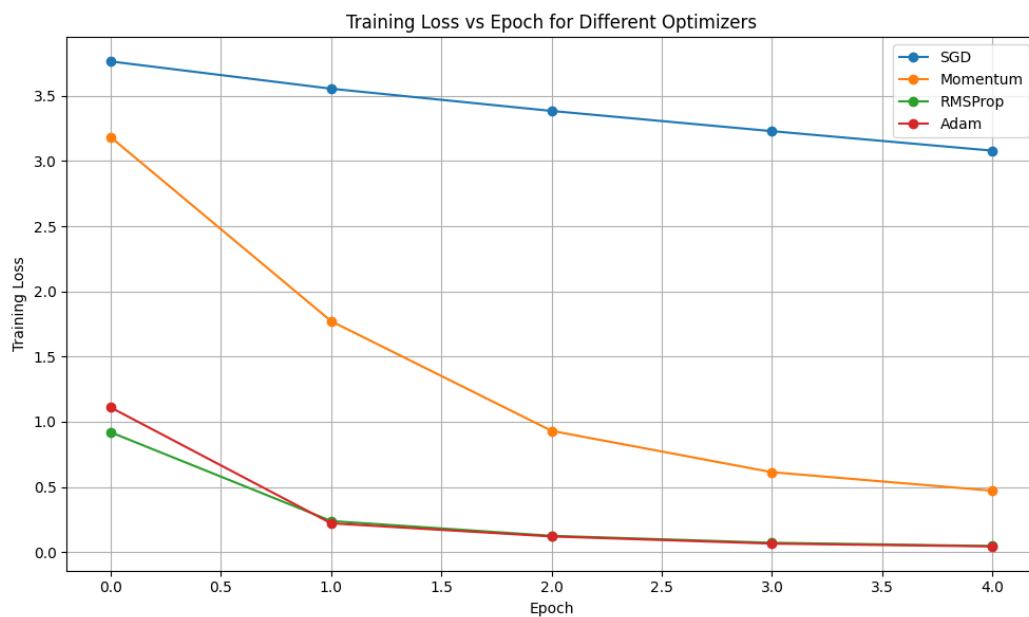


Figure 7: Training Loss for Different Optimization Algorithms

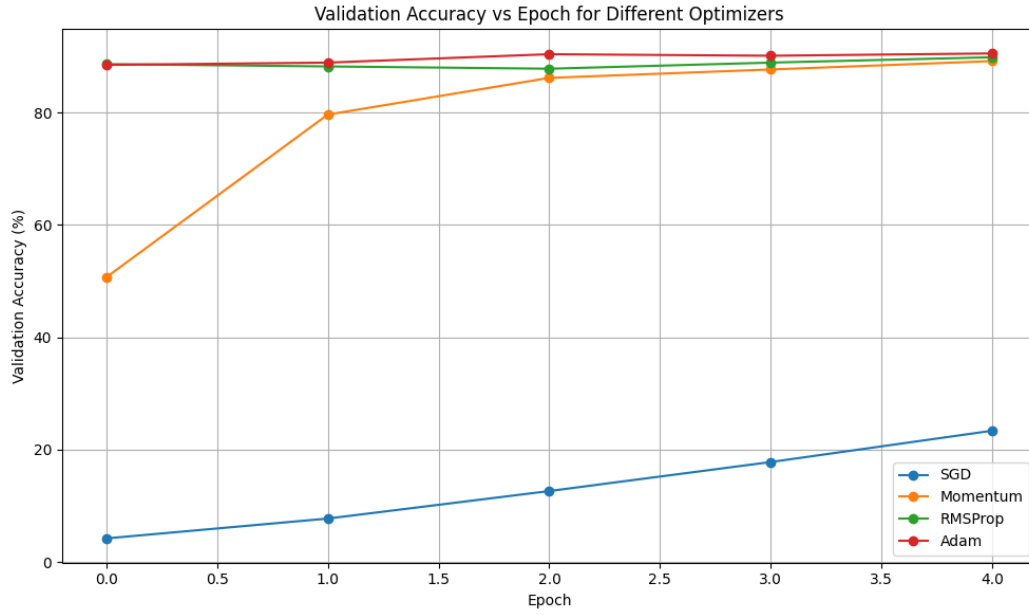


Figure 8: Validation Accuracy for Different Optimization Algorithms

The optimizer comparison is based on final loss, best validation accuracy, epoch of best validation accuracy and training time generated during runtime.

10. CNN Hyperparameter Tuning

Hyperparameters control training behaviour and can significantly affect convergence and generalization.

Hyperparameter	Values investigated
Learning Rate	0.0005, 0.0001
Batch Size	16, 24, 48
Dropout Rate	0, 0.30, 0.50
Optimizer	SGD, Momentum, RMSProp, Adam
Fine-Tuning Learning Rate	10^{-5}
Frozen / Unfrozen Base	Frozen base / selected upper layers unfrozen

During controlled hyperparameter experiments, one parameter is changed at a time while the remaining settings are held fixed.

For a convolution operation, the output dimension is

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Kernel/Filter: A small learnable matrix that extracts local features such as edges, textures and patterns.

Stride: The number of pixels by which a kernel moves across the input.

Padding: Additional pixels added around the boundary to control spatial dimensions.

Pooling: A dimensionality-reduction operation that retains important information.

Channels: The depth of an image or feature map. RGB images have three input channels.

Depthwise Separable Convolution: Separates spatial filtering from channel-wise feature combination, reducing computation compared with a standard convolution.

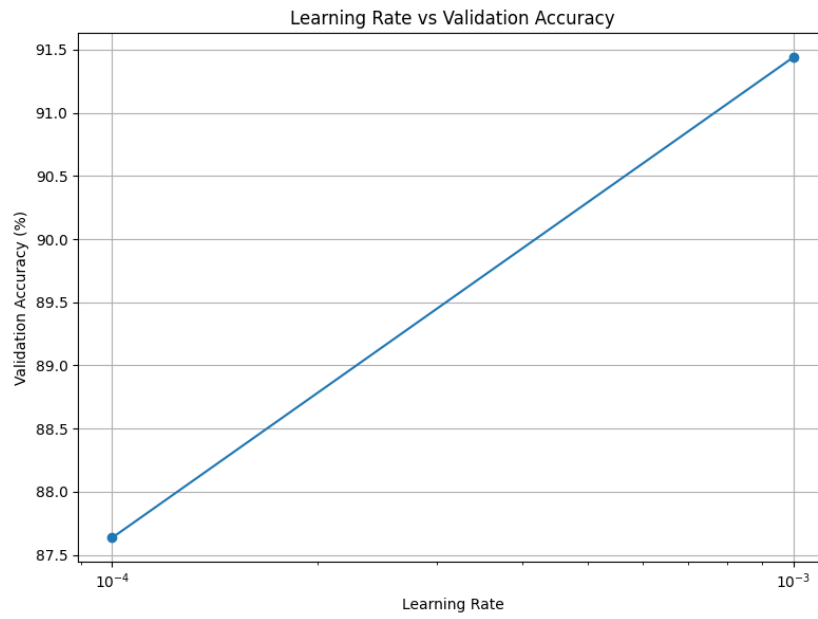


Figure 9: Effect of Learning Rate on Validation Accuracy

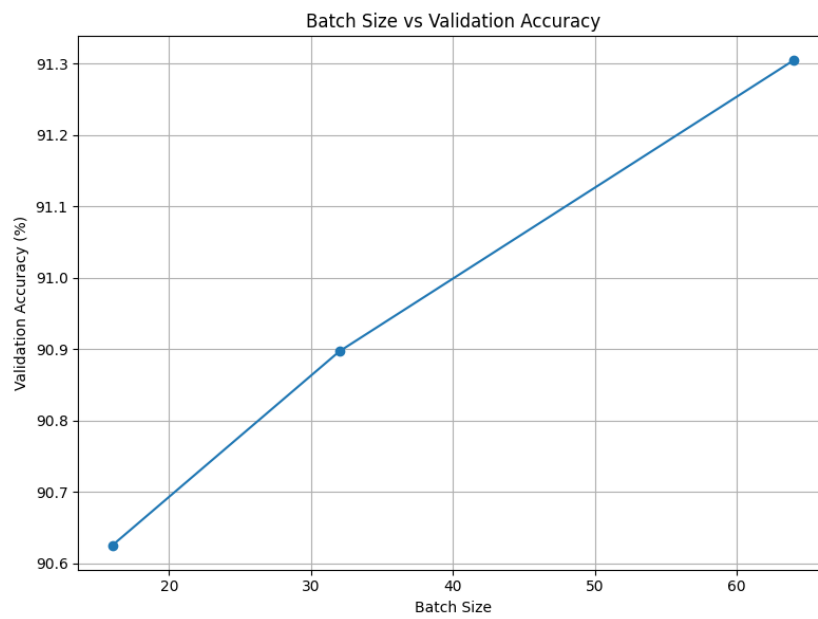


Figure 10: Effect of Batch Size on Validation Accuracy

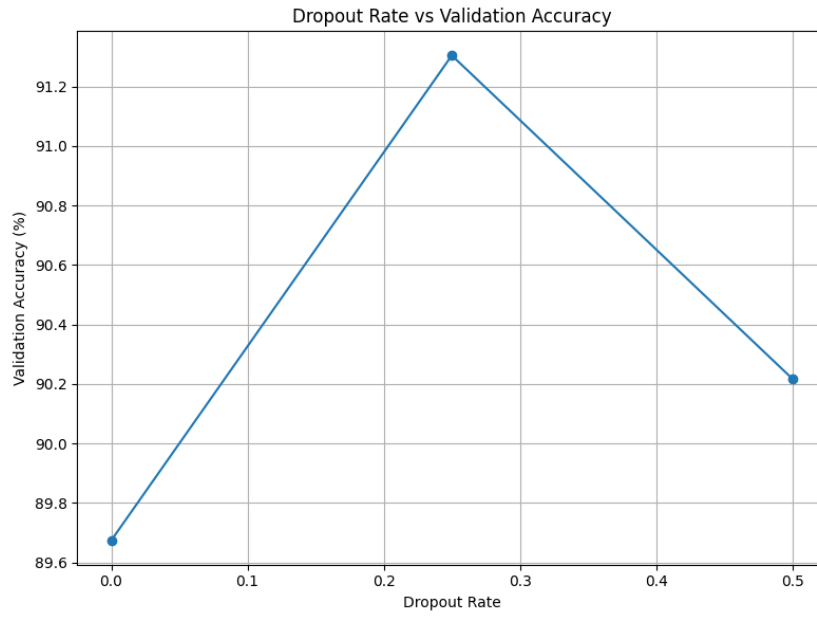


Figure 11: Effect of Dropout Rate on Validation Accuracy

11. Transfer Learning and Fine-Tuning

Transfer learning uses knowledge learned from a previously trained model and adapts it to a new classification task. In this experiment, ImageNet-pretrained MobileNetV2 is used as the base CNN.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers remain frozen and only the new classifier is trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

In the modified implementation, the last 30 selected layers are candidates for training and Batch Normalization layers remain frozen. A learning rate of 10^{-5} is used.

Aspect	Feature Extraction	Fine-Tuning
Base Network	ImageNet MobileNetV2	ImageNet MobileNetV2
Pretrained Layers	Frozen	Selected upper layers unfrozen
Trainable Parameters	Fewer	More
Learning Rate	Normal classifier LR	Smaller LR
Training Cost	Lower	Higher
Dataset Adaptation	Limited	Greater

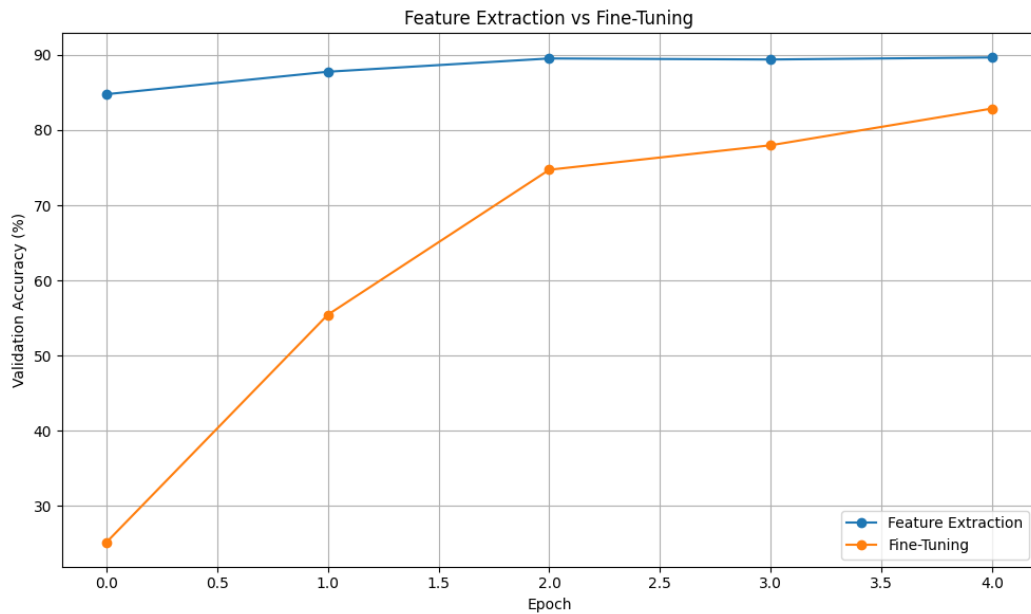


Figure 12: Validation Accuracy: Feature Extraction vs. Fine-Tuning

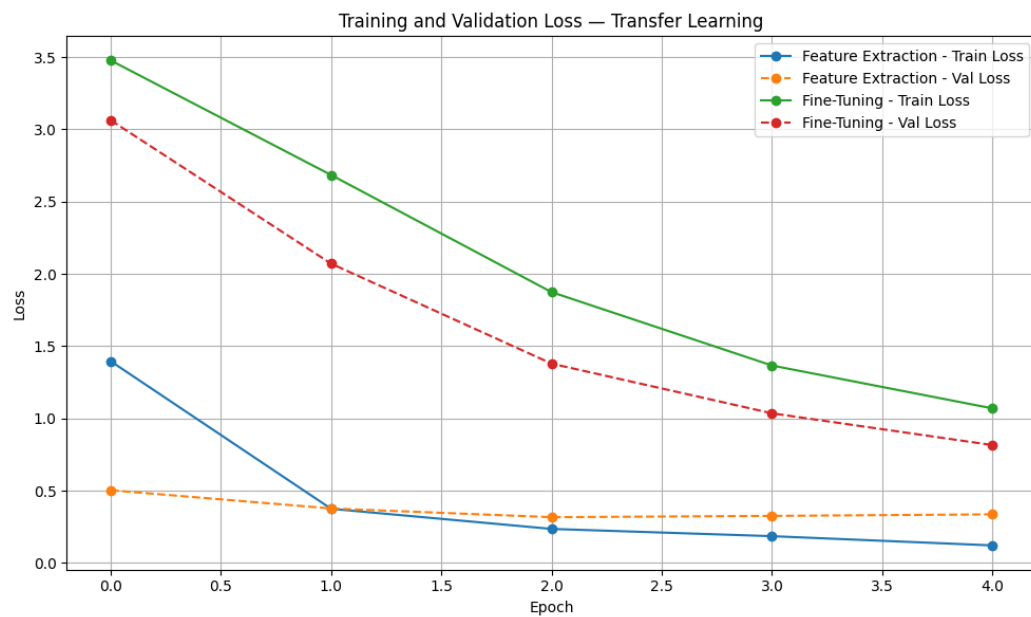


Figure 13: Training and Validation Loss Before and After Fine-Tuning

Fine-tuning normally requires a smaller learning rate because the pretrained model already contains useful representations. A smaller learning rate allows gradual task-specific adaptation.

12. K-Fold Cross-Validation

After completing the individual experiments, promising configurations are evaluated using 5-fold cross-validation.

For $K = 5$, four folds are used for training and the remaining fold is used for validation. This is repeated five times so that each fold serves as the validation fold once.

The mean validation accuracy is

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and the standard deviation is

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

The result is reported as

$$\bar{A} \pm SD.$$

The independent test set is not used for hyperparameter selection.

A key implementation improvement is memory handling. Instead of concatenating every resized image into one large NumPy array, the modified Part 2 notebook retains fold indices and filters the original TensorFlow Dataset through the input pipeline. Image resizing and preprocessing therefore occur during dataset iteration.

Configuration	Modified definition
C1	Frozen MobileNetV2 + Dense(160)
C2	Frozen base + Dense(160) + Dropout(0.30)
C3	Frozen base + Global Average Pooling + BatchNorm + Dense(160)
C4	Last 30 selected layers trainable + Dropout(0.30), LR 10^{-5}

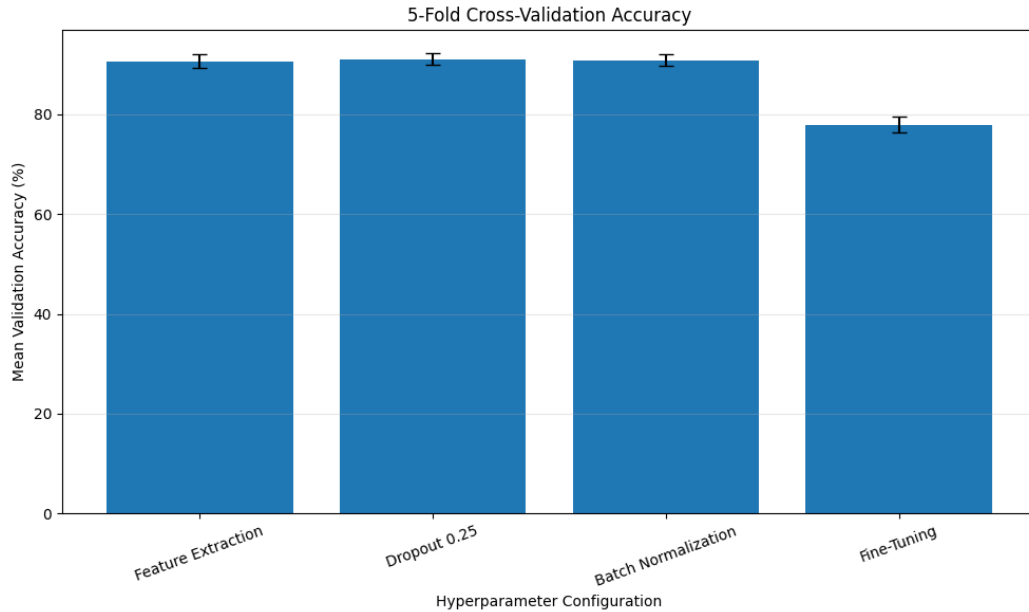


Figure 14: 5-Fold Cross-Validation Accuracy of the Evaluated Configurations

The selected configuration is determined programmatically from the actual five-fold mean validation accuracy. No fold result is fabricated or copied from the original report.

13. Final Model Evaluation

After selecting the configuration using cross-validation, the final model is rebuilt from fresh pretrained weights and trained using the selected configuration. The independent test set is evaluated only after model selection.

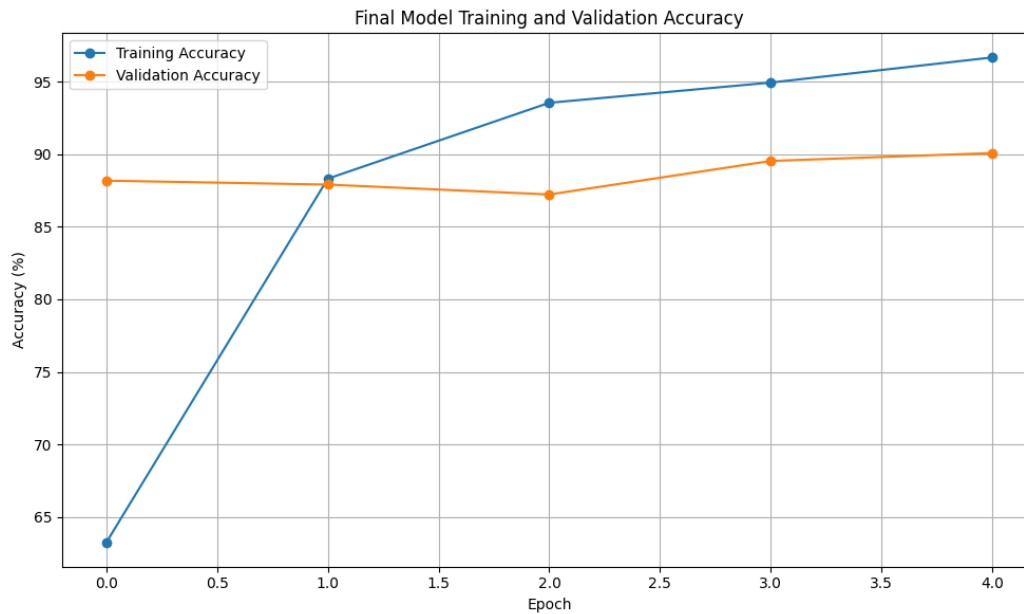


Figure 16: Final Model Training and Validation Accuracy

14. Overall Results

The final results are automatically written by the notebooks to the directory `Ex5.Modified.Outputs`.

Result file	Generated by
<code>part1.summary.csv</code>	Modified Part 1
<code>cv_results.csv</code>	Modified Part 2 – 5-fold CV
<code>final_model_results.csv</code>	Modified Part 2 – final evaluation
<code>selected_vs_additional.csv</code>	Modified Part 2 – additional exercise

This design prevents accidental use of stale values. Each table is populated from the current notebook execution.

15. Additional Exercise

Two new configurations are evaluated by changing multiple hyperparameters from the selected configuration.

Configuration	LR	Dropout	Batch	Fine-tuning
Additional C1	2×10^{-4}	0.40	16	No
Additional C2	5×10^{-5}	0.20	48	Yes

The same five folds are used for these configurations. Their mean accuracy and standard deviation are compared against the selected configuration. The notebook determines the comparison from runtime values rather than assuming which configuration performs better.

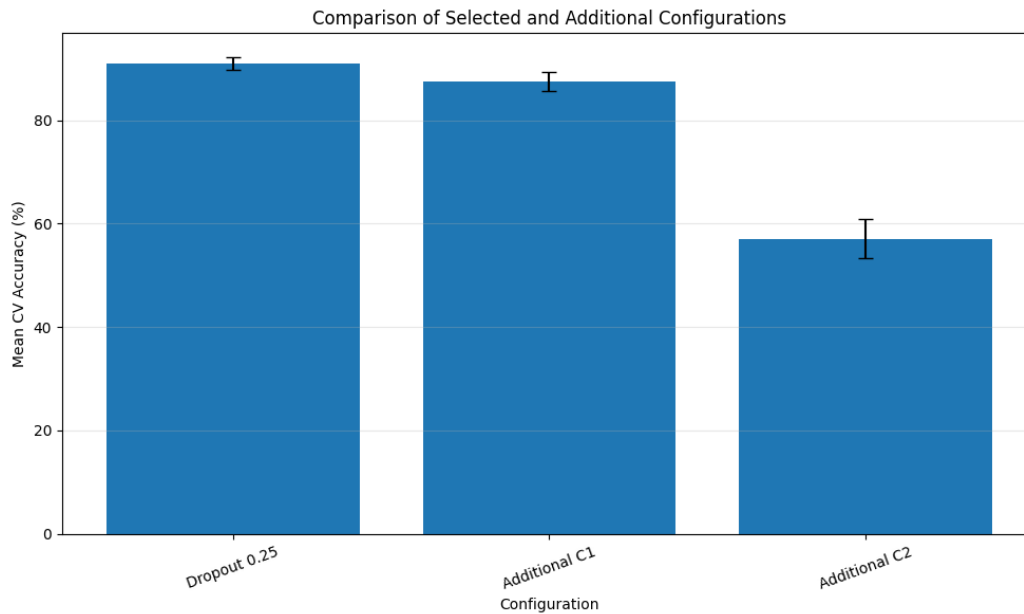


Figure 17: Comparison of Selected and Additional Configurations

16. Discussion Questions

1. Why is He initialization suitable for ReLU-based networks?

He initialization is designed to maintain an appropriate variance of activations when ReLU-based functions are used. Since ReLU sets negative activations to zero, suitable initialization helps maintain stable signal propagation.

2. Why does Batch Normalization reduce sensitivity to initialization?

Batch Normalization normalizes intermediate activations using mini-batch statistics. This keeps activations in a more stable range and reduces dependence on the exact initial weight values.

3. Why does Adam often converge faster than SGD?

Adam adapts the learning rate for individual parameters using estimates of the first and second moments of gradients. This adaptive update mechanism can provide faster and more stable convergence in many training situations.

4. Why can increasing model capacity increase overfitting?

Increasing model capacity gives a network more parameters and greater ability to represent complex patterns. If capacity is excessive relative to the available data, the network may memorize training examples rather than learn patterns that generalize.

5. Why does fine-tuning require a smaller learning rate?

A pretrained MobileNetV2 already contains useful feature representations. A large learning rate can change those weights too aggressively. A smaller learning rate permits gradual adaptation while preserving useful pretrained features.

17. Implementation Verification

The modified notebooks were checked for valid notebook structure and Python syntax of their code cells.

Notebook	Total Cells	Code Cells
DL_Lab_5_Modified_Part_1.ipynb	26	26
DL_Lab_5_Modified_Part_2.ipynb	17	17

The current document deliberately distinguishes notebook syntax verification from full TensorFlow execution. The embedded figures are retained from the supplied executed notebook outputs as visual reference; they are not claimed as newly measured results from the modified parameter set. The training jobs require TensorFlow, TensorFlow Datasets and access to the Oxford-IIIT Pet dataset. Therefore, numerical results are generated by the notebooks at runtime rather than invented in the report.

The modified implementation also addresses common failure points:

- fresh optimizer objects are created for independent experiments;
- the cross-validation pipeline avoids materializing the complete image set in a large NumPy array;
- fold statistics are calculated from actual histories;
- final test metrics are calculated from actual predictions; and
- result CSV files are written automatically.

18. Conclusion

The modified Experiment 5 implementation studies CNN initialization, regularization, Batch Normalization, optimization, hyperparameter tuning, transfer learning, fine-tuning and 5-fold cross-validation using MobileNetV2 and the Oxford-IIIT Pet dataset.

The main implementation changes include a new random seed, modified batch size, classifier width, learning rates, dropout and fine-tuning depth. The Part 2 implementation additionally improves memory usage during cross-validation by performing preprocessing through TensorFlow data pipelines rather than constructing a large image tensor in memory.

The independent test set is reserved for final evaluation. The numerical outcomes for the modified parameter set must be generated by executing the modified notebooks in a TensorFlow-enabled environment; the report does not fabricate those values.

19. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation.
6. Keras Documentation.
7. CS3807 Deep Learning Laboratory, Experiment 5 supplied specification.